```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

5. Tutorial - Lists & Abstract Data Types

Kuzey Aydınhan | 5. December 2025

Contents

1. Overview

2. Type Conversions

3. `equals` & `toString`

4. Lists & Abstract Data Types

5. Iterator

Information

Important

Register for the Übungsschein!! Deadline is 10. Dec
Campus system: https://campus.studium.kit.edu/

Dates

- There are no tutorials from 17. Dec until 06. Jan → Next week our last tutorial for year
- There are no classes from 24. Dec until 06. Jan
- The Präsenzübung is on 21. Jan
- More information under: https://sdq.kastel.kit.edu/wiki/

Last Week

Arrays

- Array lengths are fixed
- Access elements by index: `array[index]`
- Can get size information
- 0-indexed

```java
int[] array = new int[2];
int length = array.length;  // = 2
array[0] = 0;
array[1] = 1;

array[2] = 2;  // ArrayIndexOutOfBoundsException

```

Last Week

Arrays

- If array initialized (even with defaults) → length returns the size
- If not initialized → array doesn't have any size

```
int[] array;  // uninitialized
int empty = array.length;  // = 0

int[] otherArray = new int[10];  // initialized
// = [0, 0, 0, 0..]
int nonEmpty = array.length;  // = 10

```

Last Week

Encapsulation

- Encapsulation implements information hiding principle (Geheimnisprinzip)
- Attributes are hidden from outside
- Use `getters / setters` instead

Bad Code:

```
Car car = new Car("mercedes", "green", "c200");

car.name = "bmw";
car.model = "i200";
String color = car.color;

```

Correct:

```
Car car = new Car("mercedes", "green", "c200");

car.setName("bmw");
car.setModel("i200");
String color = car.getColor();

```

Last Week

Encapsulation

```java
public class Circle {
    private float radius;

    public Circle(float radius) {
        this.radius = radius;
    }

    public void setRadius(float radius) {
        this.radius = radius;
    }

    public float getRadius() {
        return this.radius;
    }

    // ...
}

```

- Make every attribute **`private`**
- Add **getters & setters** for each attribute

Motivation

- Type conversions
- `equals()` and `toString()`
- Abstract Data Types
- Lists
- Iterator

Type Conversions

Automatic Type Conversion (Widening - implicit)

Narrowing (explicit)

Type Conversions

Explicit:

- Already seen
- `(type) casting`
- **Narrowing** the type
- Careful when going from big $\rightarrow$ small

Implicit:

- Done automatically
- Only when it is safe (**widening**)

equals()

```java
public static void main(String[] args) {
    Car firstCar = new Car("sameName");
    Car secondCar = new Car("sameName");

    boolean isTrue = (firstCar == firstCar);  // = true
    boolean isFalse = (firstCar == secondCar);  // = false

    // Define .equals() for Car class
    boolean equality = firstCar.equals(secondCar);  // = true
}
```

- "==" also requires that it is the same instance / reference
- Different instances can also be same if they have the same attributes
- So we define an `.equals()` method

Strings

This is also the same with strings. You have to use `equals()` instead.

equals()

How does it look like?

- Here and only here **`instanceof`** is allowed
- **`instanceOf`** also checks if object is `null`
- When specifying `equals` you also have to override `hashCode`
  - Use the hash of **same attributes** as you use inside the `equals`
- If objects are the same, their `hashCode` must also be the same

```java
class Car {
    private String name;

    @Override
    public boolean equals(Object obj) {
        if (obj instanceof Car) {
            Car otherCar = (Car) obj;
            return this.name.equals(otherCar.name);
        }
        return false;
    }

    @Override
    public int hashCode() {
        return name != null ? name.hashCode() : 0;
    }
}
```

toString()

- Each object should be convertable to `string`
- Important properties should be returned
- This method will be called automatically when `print` or `println` called
- Default method: "name@hashCode"
  e.g. Scanner@988

```java
class Car {
    private String name;
    private String model;

    @Override
    public String toString() {
        return this.name + " " + this.model;
    }
}

```

Exercise 1

```java
class Bike {
    private String name;
    private String model;
    private Color color;
    private int year;

    @Override
    public boolean equals(Object obj) {
        // TODO: Implement me
    }

    @Override
    public int hashCode() {
        // TODO: Implement me
    }

    @Override
    public String toString() {
        // TODO: Implement me
    }
}

```

- Implement the methods
- `hashCode` should return the hash of all attributes (`Objects::hash`)
- `toString` should return a meaningful representation of the instance

Abstract Data Types

Problem

- Hard to add/remove elements from arrays
- What if size unknown?
- More complex structures needed (e.g. Graphs)

Solution:

- Define custom abstract data types
- $\rightarrow$ more flexible
- $\rightarrow$ more options
- $\rightarrow$ extendible
- e.g. Lists

Abstract Data Types

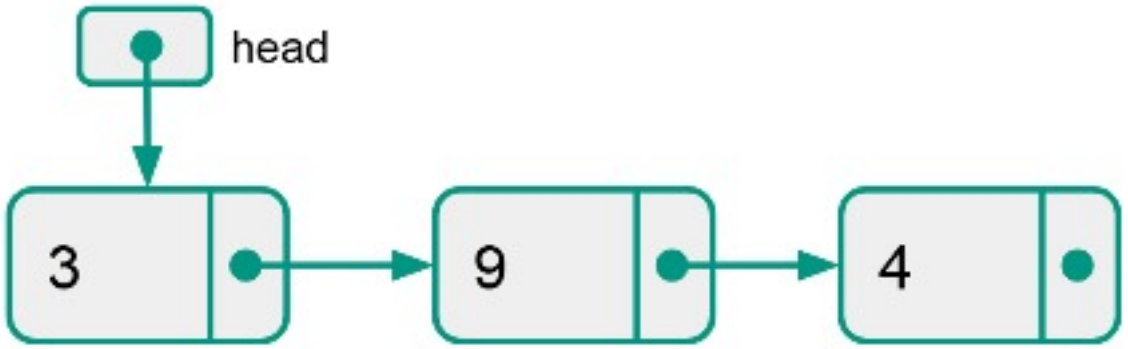
Why abstract?

- Implementation can be different
- Internal structure should be hidden (Information Hiding)
- $\rightarrow$ Usage through defined interfaces

(Linked) Lists

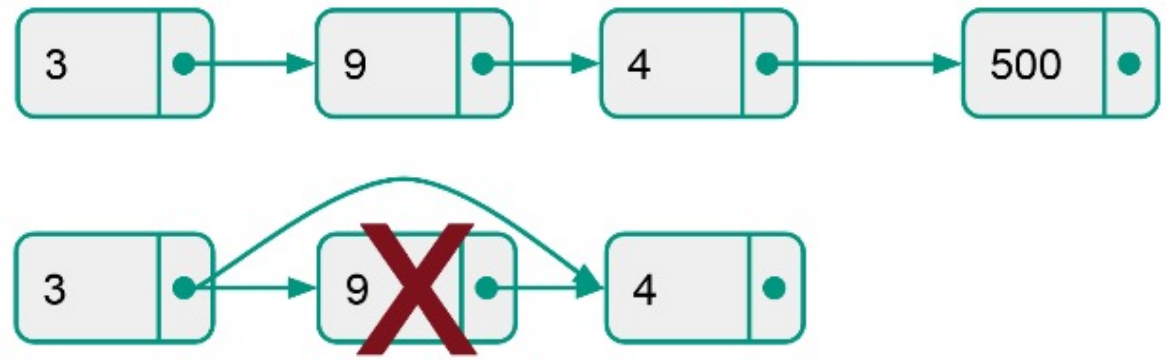
Idea

- Each item linked to the next one
- Save the information on `HEAD` (first element)
- Operations will have to be implemented
- Must be iterable



Lists

Base Operations



- `add()` : Append the list
- `delete(Element e)` : Remove the element
  - If `HEAD` is removed change `HEAD`
- `size()` : Get element amount

Lists

Other Operations

- `contains(element)`
- `containsAll(elements)`
- `indexOf(element)`
- `isEmpty()`
- `get(index)`
- `toArray()`
- ...

```java
void method(List<Integer> list, List<Integer> otherList) {

    boolean hasElement = list.contains(5);
    boolean hasElements = list.containsAll(otherList);
    int index = list.indexOf(5);  // -1 if no such element
    boolean isEmpty = list.isEmpty();
    Integer element = list.get(0);
    Integer[] array = list.toArray();
    ..
}

```

Important

Lists only accept non-primitives : use **`Integer`** not **`int`**

Lists

ArrayList / LinkedList

- Linked List
  - Complicated
  - Better for adding/removing at HEAD
  - Linked items
- Array List
  - Simpler and more efficient
  - Mostly better for any other usage
  - Dynamic array

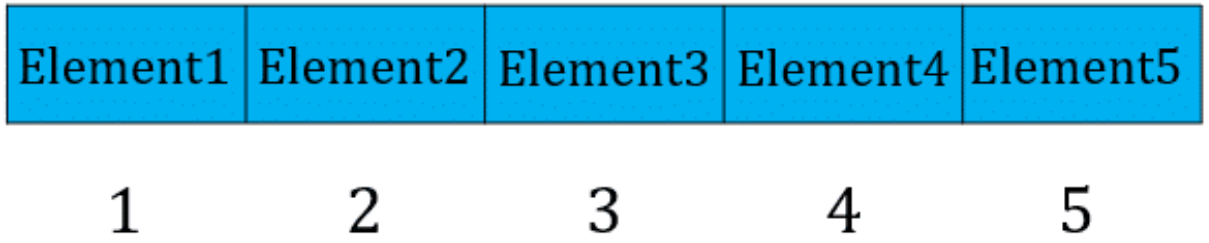
```java
public static void main(String[] args) {
    List<Player> players = new LinkedList<Player>();
    players.add(player1);
    players.add(player2);

    List<Person> people = new ArrayList<Person>();
    ..
}

```

Mostly

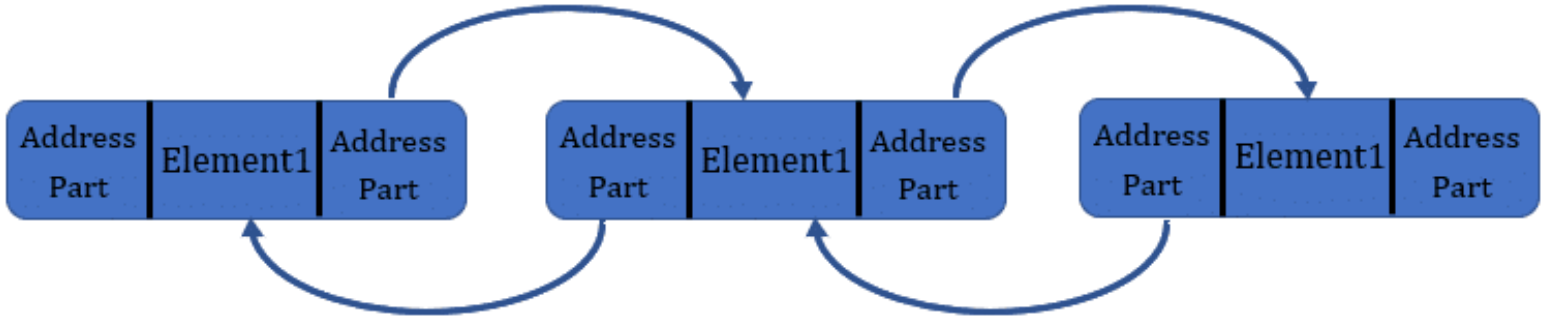
Use `ArrayList` when possible. No big difference between them

Lists

ArrayList



LinkedList



Lists vs. Arrays

Lists

- Flexible lengths
- More operations
- Access is more complicated (`get(index)`)
  - Requires iteration
- Slower

Arrays

- Fixed length
- Limited operations
- Access easier through index (`array[index]`)
- Faster

Important

Always check if `Lists` are allowed in homework

Exercise 2

- `getAllStudents:` return the list of all students
- `addStudent:` add a new student to session. Return `false` if fails
  - Keep in mind there is a capacity

```java
class Session {
    private String sessionId;
    private int capacity = 25;
    private List<Student> students = new ArrayList<>();

    public List<Student> getAllStudents() {
        // TODO: Implement me
    }

    public boolean addStudent(Student student) {
        // TODO: Implement me
    }

    public String getSessionId() { return sessionId; }
}
```

- `addSession:` add a new session. Return `false` if fails (`List::add` returns `boolean`)
- `getSession:` return session with matching id, `null` otherwise
- `clearSessions:` empties the sessions

```java
class Pool {
    private final List<Session> sessions = new ArrayList<>();

    public boolean addSession(Session session) {
        // TODO: Implement me
    }

    public Session getSession(String sessionId) {
        // TODO: Implement me
    }

    public void clearSessions() {
        // TODO: Implement me
    }
}
```

Other Abstract Types

- In general, infinite possibilities
- Some are already thought of:
  - `Sets` : Max 1 of same element
  - `Maps` : Associate objects by "key"
  - `Trees` : More in Algo 1
  - `Graphs` : More in Algo 1
  - ...

Maps

Careful

- `HashMap` : Insertion order not preserved
- `LinkedHashMap` : Order preserved
- `TreeMap` : Sorted by a property

```java
Map<String, Integer> map = new LinkedHashMap<>();
map.put("A", 1);
map.put("B", 2);
map.put("C", 3);

System.out.println(map);   // {A=1, B=2, C=3}

```

Usage

Abstracts

```java
void foo(List<String> list) {
    // Again define as list when possible
    List<String> other = new ArrayList();
    ..
}

void boo(Map<String, Integer> map) {
    // Nothing about order
    ..
}

// Only if necessary for method
void coo(LinkedHashMap<String, Integer> map) {
    // Do something with order
    ..
}
```

- Try use the most generic type when defining
- → e.g. write `List` instead of `ArrayList`

Guideline

After Homework 4: https://sdq.kastel.kit.edu/programmieren/

Usage

Collections

- Complex, editable objects such as `arrays` or `collections`(`List`, `Map` etc.) should not be exposed
- → Instead of returning the whole object, make a **shallow copy**
- Expose the copied object

```java
class Building {
    private List<Person> people;

    public List<Person> getPeople() {
        return List.copyOf(this.people);
    }
}

```

Guideline

After Homework 3: https://sdq.kastel.kit.edu/programmieren/

Iterator

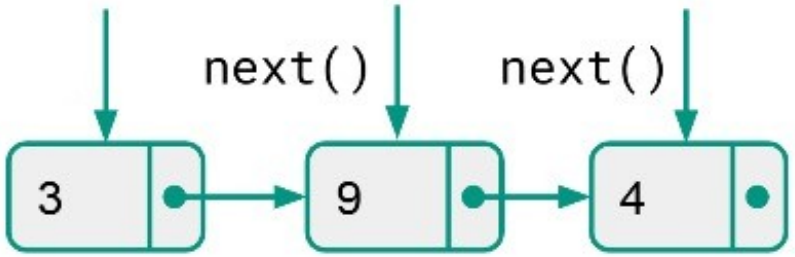
Idea

- Working with `for` loops
- Overview:
  1. Start with inital value
  2. Execute something
  3. Increase value
  4. **Break** when no element left
- **Generalized version: Iterator**
  - Usage with `for-each`
  - Must implement the interface `java.util.Iterator` (More in future)

Iterator

Implementation

- Initial value → Contstructor
- Execute something with current value
- Increase value → call `next()`
- Any element left → call `hasNext()`



Exercise 3

Data Structures

Implement the methods of a **singly linked list**.

```java
class CustomList {
    private CustomListElement head = null;
    private CustomListElement tail = null;
    private int size = 0;

    public Integer getTail() {
        // TODO: Implement me
    }

    public void insertTail(Integer value) {
        // TODO: Implement me
    }

    public Integer removeHead() {
        // TODO: Implement me
    }

    public boolean contains(Integer value) {
        // TODO: Implement me
    }
}
```

```java
class CustomListElement {
    private final Integer value;
    private CustomListElement next;

    public CustomListElement(Integer value) {
        this.value = value;
        this.next = null;
    }

    public Integer getValue() {
        return value;
    }

    public CustomListElement getNext() {
        return next;
    }

    public void setNext(CustomListElement next) {
        this.next = next;
    }
}
```

Conclusion

- Arrays
- Multidimensional arrays
- Encapsulation
- Access modifiers
- Type conversions
- `toString() & equals()`

Solution

Exercise 1

```java
class Bike {
    private static final String STRING_PATTERN =
        "Bike{name=%s, model=%s, color=%s, year=%d}";

    private String name;
    private String model;
    private Color color;
    private int year;

    @Override
    public boolean equals(Object obj) {
        if (obj instanceof Bike) {
            Bike bike = (Bike) obj;
            return Objects.equals(bike.name, this.name)
                    && Objects.equals(bike.model, this.model)
                    && this.color == bike.color
                    && this.year == bike.year;
        }
        return false;
    }

```

```java
    @Override
    public int hashCode() {
        return Objects.hash(name, model, color, year);
    }

    @Override
    public String toString() {
        return STRING_PATTERN.formatted(
            this.name, this.model, this.color, this.year);
    }
}

```

Solution

Exercise 2

```java
class Pool {
    private final List<Session> sessions = new ArrayList<>();

    public boolean addSession(Session session) {
        return sessions.add(session);
    }

    public Session getSession(String sessionId) {
        if (sessionId == null) {
            return null;
        }

        for (Session session : sessions) {
            if (Objects.equals(session.getSessionId(),
                        sessionId)) {
                return session;
            }
        }
        return null;
    }

    public void clearSessions() { sessions.clear(); }
}

```

```java
class Session {
    private String sessionId;
    private int capacity = 25;
    private List<Student> students = new ArrayList<>();

    public List<Student> getAllStudents() {
        return List.copyOf(students);
    }

    public boolean addStudent(Student student) {
        if (students.size() >= capacity) {
            return false;
        }
        return students.add(student);
    }

    public String getSessionId() {
        return sessionId;
    }
}

```

Solution

Exercise 3

```java
    public Integer getTail() {
        return tail == null ? null : tail.getValue();
    }

    public void insertTail(Integer value) {
        CustomListElement newTail =
            new CustomListElement(value);

        if (size == 0) {
            head = newTail;
        } else {
            tail.setNext(newTail);
        }
        tail = newTail;
        size++;
    }

```

```java
    public Integer removeHead() {
        if (size == 0) {
            return null;
        }

        CustomListElement oldHead = head;
        CustomListElement newHead = head.getNext();

        if (newHead == null) {
            head = null;
            tail = null;
        } else {
            head = newHead;
        }
        size--;
        return oldHead.getValue();
    }

```

Solution

Exercise 3

```java
    public boolean contains(Integer value) {
        for (CustomListElement current = head; current != null; current = current.getNext()) {
            if (Objects.equals(current.getValue(), value)) {
                return true;
            }
        }
        return false;
    }

```